
WATER JUG PROBLEM

USING BREADTH-FIRST SEARCH

Individual Project Report
Python Implementation of State-Space Search

Algorithm	Breadth-First Search (BFS)
Application	Water Jug State-Space Problem

1. Abstract

The Water Jug Problem is a classic artificial intelligence and search problem. It involves two jugs with fixed capacities and the task of measuring a required target quantity of water using only filling, emptying, and pouring operations. This project solves the problem using the Breadth-First Search (BFS) algorithm. BFS explores states level by level, ensuring that the first solution found uses the minimum number of operations. The program accepts the capacities of two jugs and a target volume as input, then displays the sequence of operations required to obtain the target.

2. Introduction

The Water Jug Problem demonstrates how real-world problems can be represented as a state-space search. Each state represents the amount of water currently available in Jug A and Jug B. From any state, valid actions can be performed, such as filling a jug, emptying a jug, or transferring water from one jug to another. BFS is used because it systematically explores all possible states and finds the shortest sequence of operations needed to reach the target quantity.

3. Problem Statement

Develop a Python program that finds a sequence of valid operations to measure a target volume of water using two jugs with different capacities. The program must determine whether a solution exists and display the steps if a solution is found.

4. Objectives

- Represent the water levels in two jugs as states.
- Generate all valid next states from a current state.
- Apply Breadth-First Search to explore the state space.
- Find the shortest sequence of operations to reach the target.
- Display each operation and the resulting jug state.
- Report when no solution exists.

5. State Representation

A state is represented using a tuple:

(a, b)

Here, a is the amount of water in Jug A and b is the amount of water in Jug B. For example, (3, 1) means Jug A contains 3 units of water and Jug B contains 1 unit of water. The initial state is (0, 0), indicating that both jugs are empty.

6. Possible Operations

For every state, the program generates the following possible operations:

- Fill Jug A completely.
- Fill Jug B completely.
- Empty Jug A.
- Empty Jug B.

- Pour water from Jug A into Jug B.
- Pour water from Jug B into Jug A.

7. Algorithm Used

Breadth-First Search uses a queue to process states in the order in which they are discovered. It first explores all states reachable in one operation, then states reachable in two operations, and so on. Since BFS explores states level by level, it guarantees the shortest solution in terms of the number of operations.

8. Working of the Program

The program begins with both jugs empty. The initial state is added to a queue and marked as visited. It repeatedly removes a state from the queue and checks whether either jug contains the target volume. If the target is not found, all possible next states are generated. States not already visited are added to the queue. A parent dictionary stores the previous state, while an action dictionary stores the operation used to reach each state. When the target is reached, the program traces backward from the final state to the initial state and reverses the result to display the steps in order.

9. Pseudocode

```
START
Read capacity of Jug A, capacity of Jug B, and target volume
Create start state as (0, 0)
Add start state to queue and mark it visited
WHILE queue is not empty:
    Remove current state from queue
    IF Jug A or Jug B contains the target:
        Reconstruct and return the solution path
    Generate all possible next states
    FOR each next state not visited:
        Mark it visited, store parent and operation, add it to queue
Return no solution
END
```

10. Example

Assume Jug A has a capacity of 4 units, Jug B has a capacity of 3 units, and the target volume is 2 units. One valid BFS solution is shown below.

Step	Operation	State (Jug A, Jug B)
Start	Initial state	(0, 0)
1	Fill Jug A	(4, 0)
2	Pour A -> B	(1, 3)
3	Empty Jug B	(1, 0)
4	Pour A -> B	(0, 1)
5	Fill Jug A	(4, 1)
6	Pour A -> B	(2, 3)

The target of 2 units is obtained in Jug A.

11. Advantages

- Finds the shortest sequence of operations.
- Avoids revisiting already explored states.
- Clearly shows every step in the solution.
- Works with different jug capacities and target values.

12. Limitations

- The number of possible states increases when jug capacities are large.
- The program checks whether the target occurs in one jug, not as the combined water in both jugs.
- Some target values may not be achievable for a given pair of jug capacities.

13. Conclusion

This project successfully solves the Water Jug Problem using Breadth-First Search. The problem is represented as a state-space graph where each jug configuration is a state and each water operation is an edge. BFS ensures that the solution obtained uses the minimum number of operations. The project demonstrates the practical use of state-space search, queues, visited-state tracking, and path reconstruction in artificial intelligence.

14. References

Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach.

Thomas H. Cormen et al., Introduction to Algorithms.

Python Documentation, collections.deque.